GPU Memory Hierarchy

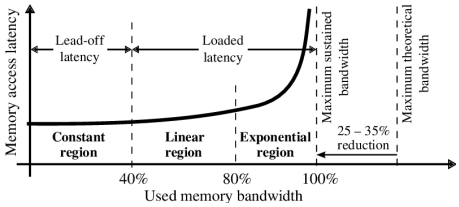
Thursday, March 12, 2026 3:22 PM

PE in Systolic Array



AAT = Hit Time + Miss Rate * Miss Latency

Mem Latency = 200 cycles



Memory Coalescing

ex. float A[N];

= f(A[tid]) ⟶ load A[tid] ⟶ fully coalesced as long as A is aligned at 4k 128 B boundary] 1 # line

ex. = A[tid/2]. (a)

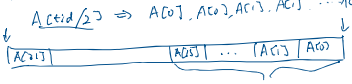
= A[tid*2] (b)

of cache lines to be accessed

(a) ? 1 (b) ? 2

Warp 0. tid 0 ~ 31

A[tid/2] ⇒ A[0], A[0], A[1], A[1] ... A[15]



A[tid+1] A aligned at 128 B boundary 2 # lines

= A[B[tid]]

A[B[tid]] =

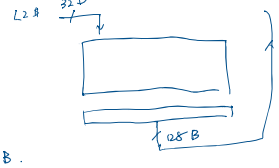
ld r1, #0(r0) // r2 = &B[0] for W0

ld r2, r1 // non coalesced, Gather operation

st r2, r1 // Scatter operation

32 x 128 B (non sector)

32 x 32 B (sectored)

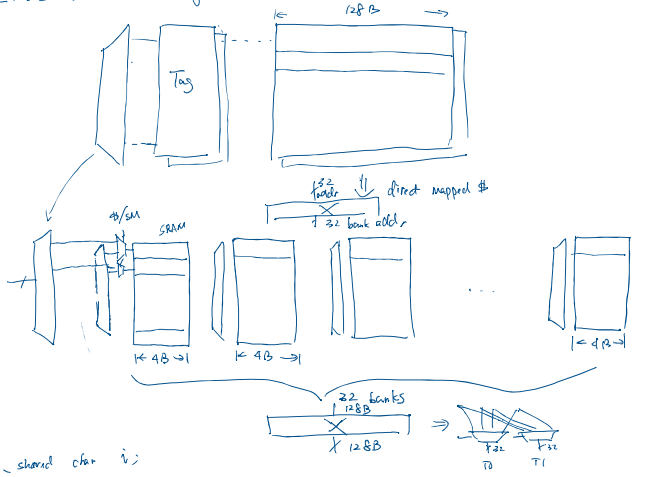


Sectored Cache (subblocking)



Sector size of 32 B.

L1 D$ & Shared Memory



__shared char i;

__shared float A[ ];



A[tid] or X[tid+1] ⇒ no bank conflict.

A[2*tid] ⟶ $ access. 2 lines.
↳ 2-way bank conflict.

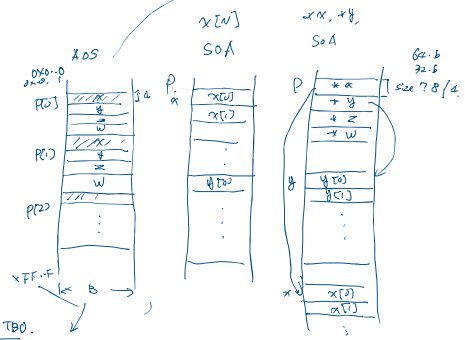
A[3*tid] in either $ access or shared memory access.
① $ access. coalesced? 3 lines.
② 32-bank shared memory. no bank conflicts.

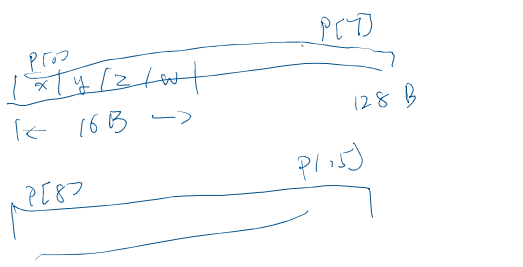


AOS vs. SOA
Array of Structures vs. Structure of arrays

```
struct {
  float x;
  float y;
  float z;
  float w;
} P[N]
```

```
struct {
  float *x;  // x[N]
  float *y;  // y[N]
  float *z;  // z[N]
  float *w;  // w[N]
} P;
```





• Global Memory vs. Shared Mem.

- Global Memory.

AOS. P[tid].x, P[tid].y, ...
SOA. P.x[tid], P.y[tid], ...
↳ w0 of TB0

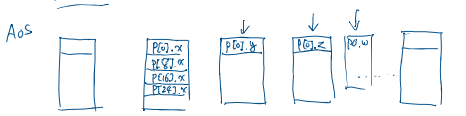
P.x[0], P.x[1], ... P.x[31]. ⟶ 1 $ line.

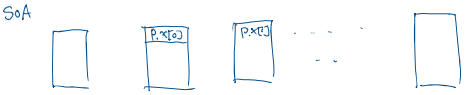
warp 0 of TB0.
P[0].x, P[1].x, P[2].x ... P[31].x
⇒ # lines.

- Shared Memory.

AoS P[tid].x ⇒ w0 of TB0. P[0].x, P[1].x, ... P[31].x ⇒ 4-way bank conflicts

SoA P.x[tid] ⇒ No Bank conflicts






```
Struct {
  int a;
  int b;
  int c;
} P[32]
```

```
Struct {
  int a[32];
  int b[32];
  int c[32];
} P.
```

for w0 of TB0?

For shared memory access, which organization has better shared memory perf? SAME.